UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Parametric Polymorphism in C++, Java, and C#

Generics vs Templates and Metaprogramming Power

Course:

Advanced Programming

Student:

Elia Leonardi

ACADEMIC YEAR 2025/2026

Contents

Chapter 1

Introduction

For the writing of the report, is use a AI local model with LM studio with qwen2.5-coder-7b-instruct, this is helpfull for the intellsense of latex and layout of the report, but it doesn't interfere with the content of the report. The verfication of the report procede in 3 steps.

1.1 Prompt Output Comparing

Every prompt on the same language produce an output that contains a fraction of the information to other prompts of the same language, this repetition helpt to make a first check of eventualy error missing or wrong information, eventualy allucinations and in generale the quality of the output.

1.2 Prompt Output Resources

Each prompt output is follow by a list of resources that are used to generate the output, as subset of the resources is used to verify the correctness of the output.

1.3 Output Script

Generated scripts are verified by running them in a controlled environment, checking for errors, and ensuring they produce the expected results. No AI model is involved in this step, as it is a manual verification process.

1.4 AI verification

An AI model is used on the resoult of the previous steps of the verification, if there is any error or missing information, the process of verification is repeated until the output is correct and complete. The AI model used for this step Claude Sonnet 5.

The following chapters contain the verification of the report, there are all the section or subsection where is find an error, missing information, imprecisions or allucina-

tions, with the corrected information and the resources used to verify the correctness All the absent section or subsection can be assumed as correct and verified.

Chapter 2

Task 1

0 Introduction

0.5 Performance

Imprecisions:

```
Homogeneous implementations generally produce smaller binaries and
↪  reduce compilation
time because generic definitions are compiled only once.
```

This statement is presented as a general rule, but it is too absolute: the reduction in compilation time holds for the type-checking and code-generation phase specific to the generic definitions themselves, but it does not account for additional compilation work introduced by erasure, such as the insertion of runtime casts, boxing/unboxing operations, or bridge methods [1]. The claim is therefore corrected as follows:

```
Homogeneous implementations generally produce smaller binaries
↪  because generic definitions
are compiled only once; compilation time is often reduced as well,
↪  although this benefit can
be partially offset by the additional work required to insert runtime
↪  casts, boxing/unboxing
operations, or bridge methods.
```

1 C++ Templates

1.1 Instantiation Time

1.1.2 Instantiation Trigger

Imprecisions:

```
In every case, code generation begins only once the complete set of
↪  arguments
is known.
```

This statement is presented as universally true for every instantiation trigger mentioned (implicit instantiation, explicit instantiation, explicit specialization), but it does not hold for *explicit instantiation declarations* (`extern template`), where the template arguments are fully known yet code generation in that translation unit is deliberately suppressed and deferred to another translation unit containing the corresponding explicit instantiation definition [16]. The claim is therefore corrected as follows:

```
In most cases, code generation begins once the complete set of
↪ arguments is
known; the exception is the explicit instantiation declaration
↪ (extern
template), which suppresses code generation in the current
↪ translation unit
even though the arguments are fully known, deferring it to a
↪ translation unit
containing the corresponding explicit instantiation definition.
```

1.1.6 Completion of the Instantiation Process

Imprecisions:

```
Instantiation terminates entirely before linking: by the time object
↪ files are
generated, every specialization required by the translation unit
↪ already exists
as compiled code, and the final linked executable contains all of
↪ them.
```

This statement correctly describes what happens within a single translation unit, but it obscures the role of the linker when the same instantiation is required by multiple translation units: each translation unit independently generates its own copy of the specialization, and it is the linker, not the compiler, that discards the duplicate copies and retains a single definition in the final executable, typically through weak symbols or COMDAT sections [8]. The claim is therefore corrected as follows:

```
Instantiation is triggered and resolved entirely during compilation:
↪ by the
time object files are generated, every specialization required by a
↪ given
translation unit already exists as compiled code. When the same
↪ instantiation
is required by multiple translation units, each one generates its own
↪ copy,
and it is the linker that discards the duplicates and retains a
↪ single
definition in the final executable.
```

1.2 Runtime Representation

noindent**Error**:

```
Hence the runtime representation of Vector<int> and Vector<double> is
↪ not a
single generic object Vector<T>, but two independent concrete types
↪ produced
before execution.
```

This sentence refers to `Vector<T>`, a type that never appears anywhere in this subsection: the only example actually shown is the function template `add<T>`, instantiated as `add<int>` and `add<double>`. The conclusion is therefore disconnected from the example that precedes it and cannot be verified against it. The claim is corrected by referring to the example actually presented:

```
Hence the runtime representation of add<int> and add<double> is not a
↪ single
generic function add<T>, but two independent concrete functions
↪ produced
before execution.
```

1.2.2 Monomorphization

Imprecisions:

```
C++ uses heterogeneous translation, commonly called monomorphization.
```

Presenting "monomorphization" as the common C++ terminology is misleading: the term originates from and is standard in the Rust community, where it names an analogous compile-time specialization process [15]. In the C++ standard and in the reference literature, the same mechanism is consistently called *template instantiation*, not monomorphization [7, 16]. The claim is therefore corrected as follows:

```
C++ uses heterogeneous translation, a process known in the C++
↪ standard and
literature as template instantiation and sometimes informally
↪ referred to,
by analogy with other languages such as Rust, as monomorphization.
```

1.2.3 Name Mangling

Imprecisions:

```
Compilers encode template arguments into symbol names through name
↪ mangling,
allowing different specializations to coexist in object files and
↪ letting
the linker distinguish entities such as function<int>() and
↪ function<double>().
```

This statement describes name mangling as a single, uniform mechanism, but the mangling scheme is not standardized across compilers: it is defined by the underlying ABI, and different ABIs produce different mangled names for the same specialization. The Itanium C++ ABI, used by GCC, Clang, and most Unix-like platforms, defines one such scheme, while MSVC on Windows uses a different, incompatible mangling scheme [8]. The claim is therefore corrected as follows:

```
Compilers encode template arguments into symbol names through name
 ↪  mangling,
allowing different specializations to coexist in object files and
 ↪  letting
the linker distinguish entities such as function<int>() and
 ↪  function<double>();
the exact mangling scheme is not standardized across compilers but is
 ↪  defined
by the underlying ABI, such as the Itanium C++ ABI used by GCC and
 ↪  Clang.
```

1.3 Performance Characteristics

1.3.1 Static Dispatch and Elimination of Runtime Overhead

Imprecisions:

```
Given template<typename T> void process(T& object) {
 ↪  object.compute(); }
called with a Matrix, the compiler resolves the call directly to
Matrix::compute(), reducing overhead and exposing further
 ↪  optimization
opportunities.
```

This statement presents call resolution as unconditionally direct, but knowing the concrete type `T` at compile time only guarantees that name lookup and overload resolution are resolved statically; it does not by itself prevent virtual dispatch. If `Matrix::compute()` is declared `virtual` within its class hierarchy, the call may still be resolved at runtime through the virtual table, even though the type `T` was fully known during template instantiation [14]. The claim is therefore corrected as follows:

```
Given template<typename T> void process(T& object) {
 ↪  object.compute(); }
called with a Matrix, the compiler resolves the type of object
 ↪  statically at
compile time; if Matrix::compute() is not virtual, the call itself is
 ↪  also
resolved directly, reducing overhead and exposing further
 ↪  optimization
```

```
opportunities, whereas a virtual compute() would still be dispatched
↪ through
the virtual table at runtime despite the type being known at compile
↪ time.
```

1.3.2 Compiler Optimization Opportunities

Imprecisions:

```
A fundamental example is function inlining: since both implementation
↪ and
argument types are known at compile time, \texttt{template<typename
↪ T> inline
T square(T x) \{ return x*x; \}} called as \texttt{square(5)} can be
↪ replaced
directly by \texttt{5*5}, removing call overhead and enabling further
↪ constant
propagation, constant folding, dead-code elimination, loop
↪ optimization, and
strength reduction.
```

This statement presents inlining and the resulting substitution as a guaranteed outcome, but the C++ standard does not require compilers to actually inline a function marked `inline`: the keyword only affects linkage rules, and actual inlining remains an optimization decision left to the compiler [7]. The claim is therefore corrected as follows:

```
A fundamental example is function inlining: since both implementation
↪ and
argument types are known at compile time, \texttt{template<typename
↪ T> inline
T square(T x) \{ return x*x; \}} called as \texttt{square(5)} can, at
↪ the
compiler's discretion, be replaced directly by \texttt{5*5}, removing
↪ call
overhead and enabling further constant propagation, constant folding,
dead-code elimination, loop optimization, and strength reduction; the
\texttt{inline} keyword only permits this substitution by affecting
↪ linkage,
without guaranteeing that the compiler will actually perform it.
```

Imprecisions:

```
Compile-time specialization also enables branch elimination via
\texttt{if constexpr}, where only the branch matching the selected
↪ type is
emitted, and improves automatic vectorization: because concrete types
↪ and
```

```
memory layouts are known, numerical templates such as an element-wise
↪ array
\texttt{add} can be compiled directly into SIMD instructions.
```

This statement presents automatic vectorization as a direct and automatic consequence of monomorphization alone, but knowing concrete types and memory layouts is necessary, not sufficient: whether SIMD instructions are actually emitted depends on the optimizer's capabilities, the compilation flags used, the absence of pointer aliasing between the operands, and the target architecture [10]. The claim is therefore corrected as follows:

```
Compile-time specialization also enables branch elimination via
\texttt{if constexpr}, where only the branch matching the selected
↪ type is
emitted, and creates the preconditions for automatic vectorization:
↪ because
concrete types and memory layouts are known, numerical templates such
↪ as an
element-wise array \texttt{add} become eligible for compilation into
↪ SIMD
instructions, although whether vectorization actually occurs still
↪ depends
on the optimizer, the compilation flags, and the absence of pointer
↪ aliasing
between operands.
```

1.3.3 Zero-Overhead Abstraction and Trade-offs

Imprecisions:

```
Therefore, template performance is generally dominated by the
↪ benefits of
compile-time specialization, while instruction cache behavior
↪ represents the
main architectural factor capable of reducing these advantages.
```

This statement generalizes as a rule what is only a common case: in codebases with extensive template usage over many distinct types, such as header-only libraries, code bloat can also increase compilation and linking time, and in some cases the resulting instruction cache pressure can outweigh the benefits of compile-time specialization rather than merely reducing them [16]. The claim is therefore corrected as follows:

```
Therefore, template performance is often dominated by the benefits of
compile-time specialization, while instruction cache behavior
↪ represents the
main architectural factor capable of reducing these advantages; in
↪ codebases
```

```
with extensive template usage over many distinct types, however, the
resulting code bloat can also affect compilation and linking time,
↪  and in
some cases can outweigh the benefits of specialization rather than
↪  merely
reducing them.
```

1.4 Type Safety

Error:

```
C++ templates adopt a reified implementation strategy based on the
↪  template
instantiation and monomorphization process described in
Section~\ref{sec:monomorphization}: type parameters are never erased,
↪  and
every instantiation is checked against a concrete, fully specialized
implementation.
```

This statement is in direct tension with how the document itself defines "reified generics" elsewhere: reification is characterized as preserving concrete type information *at runtime*, such that runtime reflection or RTTI can recover the actual instantiated types. The Runtime Representation section of this same chapter states the opposite for C++, explicitly noting the "Absence of Generic Metadata": no information about the original template or its type parameters survives as a runtime entity. Describing C++ templates as "reified" is therefore inconsistent with the standard meaning of the term in the parametric polymorphism literature, where reification specifically refers to runtime-observable type information [3]. C++ achieves static type safety through compile-time specialization into concrete, independently type-checked implementations, not through a runtime reification mechanism. The claim is therefore corrected as follows:

```
C++ templates adopt a specialization-based implementation strategy
↪  based on
the template instantiation and monomorphization process described in
Section~\ref{sec:monomorphization}: type parameters are never erased
↪  before
compile-time checking, and every instantiation is checked against a
↪  concrete,
fully specialized implementation, even though no corresponding type
information survives as a runtime entity.
```

1.5 Reflection and Runtime Type Information

Error:

```
As introduced in the overview of implementation strategies, the
↪ availability
of generic type information at runtime depends directly on whether
↪ the
implementation follows a type-erasure or a reified model. C++
↪ templates
follow the reified model described in
Section~\ref{sec:cpp_template_runtime}: template parameters are not
represented as a single runtime generic entity, and monomorphization
preserves the identity of the concrete types used during compilation.
```

This statement contradicts the conclusion of this very same section, which states that "C++ templates therefore preserve generic type information through compile-time monomorphization rather than through runtime reification." Calling the C++ strategy a "reified model" at the opening of the section and then denying runtime reification at its close is an internal inconsistency, not merely a terminological imprecision. It is also inconsistent with the Runtime Representation section's "Absence of Generic Metadata," and with the standard sense of reification in the parametric polymorphism literature, where the term denotes runtime-observable type information [3]. The claim is therefore corrected as follows:

```
As introduced in the overview of implementation strategies, the
↪ availability
of generic type information at runtime depends directly on whether
↪ the
implementation follows a type-erasure or a reified model. C++
↪ templates
follow neither model in the runtime sense: template parameters are
↪ resolved
through monomorphization described in
↪ Section~\ref{sec:cpp_template_runtime},
which preserves the identity of the concrete types used during
↪ compilation,
but this identity is not represented as a runtime generic entity once
compilation is complete.
```

1.5.2 `dynamic_cast` and Polymorphic Template Classes

Error:

```
The dynamic_cast operator provides another RTTI mechanism for
performing safe runtime conversions within polymorphic class
↪ hierarchies. A
class must contain at least one virtual function for RTTI-based
↪ dynamic
identification to be available.
```

Phrasing the virtual-function requirement as a condition for "RTTI-based dynamic identification" in general is inconsistent with the chapter's own opening example,

where `typeid(a).name()` is applied successfully to `Wrapper<int>` and `Wrapper<double>`, neither of which declares any virtual function. The requirement of at least one virtual function applies specifically to `dynamic_cast`, and to `typeid` only when it must report the dynamic (polymorphic) type of an expression rather than its static type; `typeid` on a non-polymorphic type or on a type name always succeeds and simply yields the static type [4]. The claim is therefore corrected as follows:

```
The dynamic_cast operator provides another RTTI mechanism for
performing safe runtime conversions within polymorphic class
↪ hierarchies. A
class must contain at least one virtual function for dynamic_cast to
succeed, and for typeid applied to an expression to report the
↪ dynamic type
rather than the static type known at compile time.
```

2 Java Generics

2.2 Runtime Representation

2.2.4 Bridge Methods

Error:

```
Erasure can break overriding relationships at the JVM level. Given
Node<T>.get():T overridden by StringNode.get():String,
erasure reduces the parent signature to get():Object; the compiler
therefore generates a synthetic bridge method Object get() { return
get(); } to preserve polymorphic dispatch.
```

Representing the bridge method as `Object get()  return get();` in Java source syntax is misleading: read literally, it is an infinite recursive call to itself, and two methods differing only in return type cannot coexist as Java source in the first place. The bridge method is a compiler-generated, bytecode-level construct, distinguishable from the covariant override only by its method descriptor (`()Ljava/lang/Object;` versus `()Ljava/lang/String;`) [12]; it was introduced specifically to preserve virtual dispatch under erasure, invoking the specific covariant implementation rather than itself [2]. The claim is therefore corrected as follows:

```
Erasure can break overriding relationships at the JVM level. Given
Node<T>.get():T overridden by StringNode.get():String,
erasure reduces the parent signature to get():Object; the compiler
therefore generates a synthetic bridge method, distinguishable from
↪ the
covariant override only at the bytecode level by its descriptor
()Ljava/lang/Object;, which invokes the specific String get()
↪ implementation
and returns its result as Object, in order to preserve polymorphic
↪ dispatch
through the erased parent signature.
```

2.3 Performance Characteristics

2.3.5 Adaptive JIT Optimization

Error:

```
The HotSpot JVM mitigates these costs through runtime profiling:
devirtualization, inline caching, and escape analysis (enabling
↪ scalar
replacement of non-escaping allocations) recover some of the
↪ performance
lost to erasure. These optimizations remain speculative and
profile-dependent, unlike the ahead-of-time specialization guaranteed
↪ by
C++ monomorphization.
```

Escape analysis and scalar replacement do not generally recover performance lost to erasure: they eliminate heap allocation only for objects, including boxed primitives, that the JIT proves do not escape the enclosing method or compilation unit. This addresses a narrow subset of the boxing overhead described earlier in the section, but it does not restore contiguous, specialized storage for generic containers such as `ArrayList`, whose internal representation remains an array of `Object` references regardless of JIT optimization [12]. Presenting scalar replacement as recovering performance "lost to erasure" in general overstates its scope. The claim is therefore corrected as follows:

```
The HotSpot JVM mitigates some of these costs through runtime
↪ profiling:
devirtualization and inline caching reduce dispatch overhead, while
↪ escape
analysis enables scalar replacement of non-escaping boxed values,
eliminating their allocation in specific, provably local cases. These
optimizations reduce boxing overhead in favorable cases but do not
↪ restore
the contiguous, specialized storage of generic containers, which
↪ retain
their reference-based internal representation regardless of JIT
optimization, unlike the ahead-of-time specialization guaranteed by
↪ C++
monomorphization.
```

2.4 Type Safety

2.4.4 Comparison with C++ Reification

Error:

```
C++ templates preserve type information through reification: each
instantiation is checked against its concrete type directly, as in
```

```
Box<int> box; box.set("text");, rejected because int does
not accept a string. Java instead verifies every substitution before
erasure removes the parameter. The distinction is therefore not
↪ whether
type safety is provided, but at which stage type information is
consumed: Java achieves safety through static verification prior to
erasure, while C++ achieves safety through compile-time reification
↪ of
concrete types that persist into the generated representation.
```

This repeats, in the Java chapter, the same terminological error already identified and corrected in the C++ chapter's Reflection and Runtime Type Information section, where "reification" was shown to be inconsistent with the C++ chapter's own conclusion that generic type information is preserved "through compile-time monomorphization rather than through runtime reification." Reification, in the parametric polymorphism literature, specifically denotes runtime-observable type information [3]; C++ achieves type safety through compile-time specialization checked independently for each instantiation, with no corresponding runtime entity surviving into execution. Repeating this mischaracterization here also introduces a cross-chapter inconsistency: the C++ chapter, once corrected, explicitly denies runtime reification, while this section asserts it as the mechanism underlying C++ safety. The claim is therefore corrected as follows:

```
C++ templates preserve type information through compile-time
↪ specialization:
each instantiation is checked against its concrete type directly, as
↪ in
Box<int> box; box.set("text");, rejected because int does
not accept a string. Java instead verifies every substitution before
erasure removes the parameter. The distinction is therefore not
↪ whether
type safety is provided, but at which stage type information is
consumed: Java achieves safety through static verification prior to
erasure, while C++ achieves safety through compile-time
↪ specialization of
concrete types, checked independently for each instantiation, with no
corresponding generic entity surviving into the running program.
```

2.5 Reflection and Runtime Type Information

2.5.4 Comparison with C++ Template RTTI

Error:

```
C++ monomorphization generates an independent concrete type--and
independent RTTI metadata--for every specialization, so typeid and
dynamic_cast can distinguish Vector<int> from Vector<double>
↪ directly.
```

```
Java's erasure model instead collapses all parameterizations into a
↪ single
Class object, so reflection exposes only the identity of the generic
declaration after erasure, never the parameterized instantiation
↪ present in
the source. This contrast illustrates the underlying trade-off: C++
reification preserves runtime type identity per specialization, while
↪ Java
erasure minimizes runtime type diversity at the cost of that same
visibility.
```

This is the third occurrence in the document of the same terminological error already corrected in the C++ chapter's Reflection and Runtime Type Information section and in the Java chapter's Type Safety section: describing the C++ mechanism as "reification" contradicts the C++ chapter's own conclusion that generic type information is preserved "through compile-time monomorphization rather than through runtime reification." Reification, in the parametric polymorphism literature, denotes runtime-observable type information [3]; the RTTI metadata described here is a per-specialization by-product of monomorphization and polymorphic object layout, not a general runtime reification mechanism, as already established in the C++ RTTI section itself. The claim is therefore corrected as follows:

```
C++ monomorphization generates an independent concrete type--and
independent RTTI metadata--for every specialization, so typeid and
dynamic_cast can distinguish Vector<int> from Vector<double>
↪ directly.
Java's erasure model instead collapses all parameterizations into a
↪ single
Class object, so reflection exposes only the identity of the generic
declaration after erasure, never the parameterized instantiation
↪ present in
the source. This contrast illustrates the underlying trade-off: C++
monomorphization preserves runtime type identity per specialization
↪ as a
by-product of compile-time code generation, while Java erasure
↪ minimizes
runtime type diversity at the cost of that same visibility.
```

3 C# Generics

3.2 Runtime Representation

3.2.3 Value-Type Specialization

Imprecisions:

```
Value types have distinct memory layouts, so the JIT generates
↪ specialized
code per instantiation
```

This statement attributes specialization to the value-type nature itself rather than to the actual cause: it is the distinct size and memory layout of each value type that forces the JIT to generate separate native code, not the value/reference distinction as such [9]. The claim is therefore corrected as follows:

```
Because each value type has its own size and memory layout, the JIT
generates separately specialized native code for each such
↪ instantiation:
JIT(List<int>) = NativeCode_int, JIT(List<double>) =
↪ NativeCode_double,
each operating directly on the corresponding representation.
```

3.2.6 Runtime Type Identity

Error:

```
C++ specializations may participate in RTTI as concrete types, but
↪ the
original template definition itself is not preserved as a runtime
↪ entity.
```

This statement is inconsistent with the corrected C++ Reflection and Runtime Type Information section, where RTTI-based dynamic identification (dynamic_cast, and typeid reporting the dynamic type) was shown to require at least one virtual function in the class hierarchy [4]; presenting RTTI participation here as a general property of C++ specializations omits that condition. The claim is therefore corrected as follows:

```
C++ specializations can participate in RTTI as concrete types only
↪ within
polymorphic class hierarchies (i.e., containing at least one virtual
function); the original template definition itself is never preserved
↪ as
a runtime entity, regardless of polymorphism.
```

3.2.4 Reference-Type Code Sharing

Imprecisions:

```
Because reference types share a uniform object-reference
↪ representation,
the CLR can reuse a single implementation, NativeCode_reference,
↪ across
instantiations such as List<string> and List<object>, supplying
type-specific information through runtime execution-engine structures
rather than duplicating code.
```

This statement presents code sharing for reference types as a universal property of the CLR as an abstract specification, but this canonicalization strategy is a design choice of specific CLR implementations, described as such by Kennedy and Syme [9] for the Microsoft CLR, and is not mandated by the ECMA-335 standard itself [6]. The claim is therefore corrected as follows:

```
Because reference types share a uniform object-reference
↪ representation,
CLR implementations such as the Microsoft CLR can reuse a single
↪ compiled
implementation, NativeCode_reference, across instantiations such as
List<string> and List<object>, supplying type-specific information
through runtime execution-engine structures rather than duplicating
↪ code.
```

3.3 Performance Characteristics

3.3.1 Value-Type Instantiations

Imprecisions:

```
execution cost approximates T_value=N(C_load+C_operation), closer to
specialized C++ code than to Java's boxed collections.
```

This statement presents an aggregate per-collection cost model (T_{value}, scaled by N elements) as directly comparable to the per-operation cost model given later for Java (T_{Java}), without making the difference in scale explicit; this can mislead the reader into treating the two expressions as quantitatively equivalent when they measure different units of work. The claim is therefore corrected as follows:

```
execution cost across the whole collection approximates
T_value=N(C_load+C_operation), with per-element cost closer to
↪ specialized
C++ code than to Java's boxed collections.
```

3.3.2 Reference-Type Instantiations

Imprecisions:

```
though, unlike Java, no boxing of primitives is ever required since
↪ C#
retains explicit runtime generic support.
```

This statement is placed within the discussion of reference-type instantiations, where boxing is not at issue in any of the three languages compared, since reference types are never boxed; the actual contrast with Java concerns value-type instantiations, discussed in the preceding subsection, not the reference-type case under discussion here [9]. The claim is therefore corrected as follows:

```
though C\#'s avoidance of boxing, unlike Java, applies specifically
↪ to
value-type instantiations such as those discussed above, not to
reference-type instantiations, where none of the three languages
↪ requires
boxing.
```

3.4 Type Safety

3.4.1 Static Checking of Generic Parameters

Error:

```
Given class Box<T> { T value; void Set(T v); T Get(); }, the compiler
verifies that all values passed to Set match the chosen
↪ instantiation:
Box<int> numbers; numbers.Set("text"); is rejected because Box<int>
requires every stored value to be an integer.
```

The class declaration as written is not valid C# syntax: method members of a class require a body (or an expression-bodied definition), and semicolon-terminated signatures such as `void Set(T v);` are only permitted in interface or abstract member declarations, not in a concrete class [5]. The claim is therefore corrected as follows:

```
Given class Box<T> { private T value; public void Set(T v) { value =
↪ v; }
public T Get() { return value; } }, the compiler verifies that all
↪ values
passed to Set match the chosen instantiation: Box<int> numbers = new
Box<int>(); numbers.Set("text"); is rejected because Box<int>
↪ requires
every stored value to be an integer.
```

3.4.4 Reification and Preserved Type Information

Imprecisions:

```
Because C# is reified, Box<int> and Box<string> remain distinct
constructed types whose invariants cannot cross-contaminate.
```

This statement attributes the compile-time rejection shown earlier (Box<int>.Set("text")) to reification, but static type checking of generic usage occurs during compilation regardless of the later runtime representation; reification instead guarantees that the distinction between constructed types such as Box<int> and Box<string> also persists as a runtime entity, as already established in the Runtime Representation section [9]. The claim is therefore corrected as follows:

```
Box<int> and Box<string> are checked as distinct constructed types
↪  already
at compile time, independently of reification; because C# is reified,
this distinction also persists at runtime, so their invariants cannot
cross-contaminate at either stage.
```

3.5 Reflection and Runtime Type Information

3.5.3 Comparison with C++ RTTI

Error:

```
C++ monomorphization resolves template parameters at compile time;
standard RTTI (typeid, std::type_info) identifies the resulting
↪  concrete
type but cannot query the original template argument as runtime
↪  metadata
-- the template/parameter relationship exists only at compile time.
```

This statement presents RTTI-based identification of the concrete instantiated type as a general, unconditional capability of typeid, but as established in the C++ chapter's Reflection and Runtime Type Information section, typeid reports the dynamic (polymorphic) type only when the class hierarchy contains at least one virtual function; otherwise it always yields the static type known at compile time [4]. The claim is therefore corrected as follows:

```
C++ monomorphization resolves template parameters at compile time;
standard RTTI (typeid, std::type_info) can identify the resulting
↪  concrete
type only when the class hierarchy is polymorphic (i.e., contains at
↪  least
one virtual function), and even then cannot query the original
↪  template
argument as runtime metadata -- the template/parameter relationship
↪  exists
only at compile time.
```

3.5.5 Comparative Summary

Imprecisions:

```
C++ resolves parameters at compile time, leaving RTTI to identify
↪  only
concrete specialized types
```

This statement omits the precondition, already established in the C++ chapter, that RTTI-based dynamic type identification requires a polymorphic class hierarchy; without at least one virtual function, typeid simply yields the statically known type rather than performing any runtime identification [4]. The claim is therefore corrected as follows:

```
C++ resolves parameters at compile time, leaving RTTI able to
↪  identify only
concrete specialized types, and only within polymorphic class
↪  hierarchies
```

Chapter 3

Task 2

Error:

```
Due to type erasure, generic type parameters are generally not
available in ordinary runtime objects, reducing the ability to
↪  perform
type-based compile-time transformations.
```

This statement inverts cause and effect: type erasure is a runtime-level consequence of Java's generic implementation, not a mechanism that acts on compile-time transformation capability. The actual reason Java cannot perform type-based compile-time transformations, as the chapter itself establishes in the "Bounded Polymorphism in Java" section, is that Java generics are implemented through erasure rather than instantiation-time substitution, so no specialized version of generic code is ever generated per concrete type; the unavailability of generic type parameters in runtime objects is a downstream effect of this same implementation choice, not its cause [1]. The claim is therefore corrected as follows:

```
Because Java generics are implemented through type erasure rather
↪  than
instantiation-time substitution, no specialized version of generic
↪  code is
generated per concrete type, which prevents type-based compile-time
transformations; as a further consequence of the same erasure
↪  process,
generic type parameters are also generally unavailable in ordinary
↪  runtime
objects.
```

4.1 Compile-Time Computation in C++

4.1.1 Recursive Template Metaprogramming

Imprecisions:

```
The same mechanism applies to computations with multiple recursive
dependencies, such as the Fibonacci sequence: [...] Instantiating
Fibonacci<10>::value generates the dependency tree implementing
F(n) = F(n-1) + F(n-2) until reaching the terminating
↪  specializations,
yielding the constant 55 before code generation.
```

Presenting the Fibonacci case as governed by "the same mechanism" as the Factorial example obscures a relevant structural difference: unlike the linear instantiation chain produced by Factorial, the naive recursive expansion of Fibonacci<N> requires each intermediate specialization (e.g. Fibonacci<8>) to be reached through multiple independent paths in the dependency tree, which would in principle cause redundant instantiation work; this is mitigated only because the compiler caches each distinct template specialization once instantiated, not because the underlying recursive structure avoids duplication [16]. The claim is therefore corrected as follows:

```
A related but structurally different case is the Fibonacci sequence,
↪  whose
naive recursive definition requires each intermediate value to be
↪  reached
through multiple paths in the dependency tree, unlike the single
↪  linear
chain produced by Factorial: [...] Instantiating Fibonacci<10>::value
generates this dependency tree implementing F(n) = F(n-1) + F(n-2)
↪  until
reaching the terminating specializations, yielding the constant 55
↪  before
code generation; redundant re-instantiation of shared intermediate
↪  values
such as Fibonacci<8> is avoided only because the compiler caches each
distinct specialization once it has been instantiated.
```

4.2 Compile-Time Overload Selection: SFINAE and Concepts

```
#include <type_traits>
template <typename T> requires std::integral<T>
void process(T value) { std::cout << "Integral: " << value; }

template <typename T> requires std::floating_point<T>
void process(T value) { std::cout << "Floating-point: " << value; }
```

The concepts `std::integral` and `std::floating_point` are declared in the `<concepts>` header, not in `<type_traits>`: the latter provides the type traits `std::is_integral` and `std::is_floating_point` used in the preceding SFINAE

example, not the lowercase concepts of the same name used here [7]. As written, the code would not compile with the single include shown. The claim is therefore corrected as follows:

```
#include <concepts>
template <typename T> requires std::integral<T>
void process(T value) { std::cout << "Integral: " << value; }

template <typename T> requires std::floating_point<T>
void process(T value) { std::cout << "Floating-point: " << value; }
```

4.3 Variadic Templates and Related C++ Techniques

4.3.1 Template Parameters

```
template <typename T, template <typename> class Container>
class Stack {
    Container<T> data;
public:
    void push(const T& value) { data.push_back(value); }
};

Stack<int, std::vector> a;
Stack<int, std::list>   b;
```

The template template parameter `Container` is declared with a single type parameter (`template <typename> class`), but `std::vector` and `std::list` are declared in the standard library with two template parameters, the element type and an allocator defaulted to `std::allocator<T>` [7]. Prior to the relaxed matching rules for default template arguments in template template parameters, and inconsistently across compilers even after their adoption, this arity mismatch causes the code as written to fail to compile; the example does not account for this discrepancy. The claim is therefore corrected as follows:

```
template <typename T, template <typename, typename...> class
↪  Container>
class Stack {
    Container<T> data;
public:
    void push(const T& value) { data.push_back(value); }
};

Stack<int, std::vector> a;
Stack<int, std::list>   b;
```

4.4 Bounded Polymorphism in Java: Constraints Without Substitution

Error:

```
Since no specialized version of generic code is generated per
↪  concrete
type, a method such as

public static <T> void process(T value) {
    // cannot select an implementation based on properties of T
}

cannot be compiled into distinct versions depending on T.
```

This code example does not demonstrate the limitation it is meant to illustrate: the method body contains no operation depending on properties of `T` at all, only a comment stating the claim in prose. A reader cannot verify the limitation against this code, since there is nothing in it to fail or succeed. The following section ("Instantiation-Time Checking") already gives a concrete, verifiable example of this exact limitation (`value.serialize()` failing to compile in Java), making this fragment both unverifiable and redundant. The claim is therefore corrected by replacing the empty example with a concrete conditional operation that actually fails to compile:

```
Since no specialized version of generic code is generated per
↪  concrete
type, a method such as

public static <T> void process(T value) {
    if (value instanceof Integer) {
        // still cannot call Integer-specific methods on 'value'
        // without an explicit cast, since the declared type of
        // value remains T, not Integer
    }
}

cannot be compiled into distinct versions depending on T: even after
↪  a
runtime instanceof check, the static type of value remains T, so any
type-specific member access still requires an explicit cast performed
↪  by
the programmer, not a compiler-generated specialization.
```

4.5 Constrained Generics in C#: Approximating Type Traits

Error:

```
the closest C# approximation constrains the parameter directly:

static T Add<T>(T a, T b) where T : INumber<T>
{
    return a + b;
}

Static abstract interface members extend this further into a
↪  trait-like
contract:

public interface IAddable<TSelf> where TSelf : IAddable<TSelf>
{
    static abstract TSelf operator +(TSelf x, TSelf y);
}
```

This presents `INumber<T>` as a simpler or conceptually prior approximation that static abstract interface members subsequently "extend further," but this ordering is inverted: `INumber<T>` is itself a .NET 7 standard-library interface built directly on top of static abstract interface members, using `static abstract` operators internally to express arithmetic constraints [11]. It is an applied instance of the same mechanism described afterward, not a distinct or earlier approximation. The claim is therefore corrected as follows:

```
the closest C# approximation constrains the parameter directly, using
↪  the
standard-library INumber<T> interface introduced together with static
abstract interface members in .NET 7:

static T Add<T>(T a, T b) where T : INumber<T>
{
    return a + b;
}

This works because INumber<T> is itself defined in terms of static
abstract interface members, which allow a trait-like contract to be
expressed directly:

public interface IAddable<TSelf> where TSelf : IAddable<TSelf>
{
    static abstract TSelf operator +(TSelf x, TSelf y);
}
```

4.6 Instantiation-Time Checking and Structural Capability Detection

Error:

```
interface ISerializable { void Serialize(); }

static void Process<T>(T value) where T : ISerializable
    => value.Serialize();
```

Declaring a custom interface named `ISerializable` collides with the real, well-known `System.Runtime.Serialization.ISerializable` interface in the .NET Base Class Library, which declares a completely different member, `GetObjectData(SerializationInfo, StreamingContext)`, not a parameterless `Serialize()` method [6]. Reusing this name for an unrelated, incompatible interface in a didactic example risks being mistaken for the standard interface by a reader familiar with the .NET framework. The claim is therefore corrected as follows:

```
interface IManualSerializable { void Serialize(); }

static void Process<T>(T value) where T : IManualSerializable
    => value.Serialize();
```

4.7 External Metaprogramming Mechanisms in Java and C#

4.7.1 Java: Reflection, Annotation Processing, and Code Generation

Imprecisions:

```
Compile-time generation is instead handled by annotation processors
↪  (Java
Annotation Processing API), which run during compilation, analyze
annotated source elements through the compiler's language model API,
↪  and
generate auxiliary source files. Libraries such as Lombok extend this
through direct AST manipulation, injecting methods or constructors
↪  that
developers would otherwise write by hand.
```

This presents Lombok's AST manipulation as a straightforward extension of the standard Annotation Processing API, but the standard API is designed only to generate new auxiliary source files, not to modify the AST of existing ones. Lombok achieves in-place AST manipulation by relying on internal, unsupported compiler APIs (`com.sun.tools.javac`) rather than the public annotation processing mechanism described in the same sentence, which is why modern JDK versions require explicit `-add-opens` flags to permit this access [13]. The claim is therefore corrected as follows:

```
Compile-time generation is instead handled by annotation processors
↪ (Java
Annotation Processing API), which run during compilation, analyze
annotated source elements through the compiler's language model API,
↪ and
generate auxiliary source files. Libraries such as Lombok achieve a
similar effect through direct AST manipulation, injecting methods or
constructors that developers would otherwise write by hand; unlike
standard annotation processors, Lombok relies on internal,
↪ unsupported
compiler APIs to modify existing source rather than only generating
↪ new
files, which is why modern JDK versions require explicit
↪ module-access
flags to permit it.
```

Bibliography

[1] Gilad Bracha. Generics in the java programming language, 2004.

[2] Gilad Bracha. Generics in the java programming language. Technical report, Sun Microsystems, 2004.

[3] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.

[4] cppreference.com. *C++ Language Reference: RTTI and typeid.* cppreference.com, 2025. Online reference for C++ RTTI mechanisms, std::type_info, typeid, and dynamic_cast.

[5] Ecma International. Ecma-334: C# language specification, 2023.

[6] Ecma International. Ecma-335: Common language infrastructure (cli), 2024.

[7] ISO/IEC JTC 1/SC 22/WG 21. ISO/IEC 14882:2024 Programming Languages — C++, 2024. International Standard.

[8] Itanium C++ ABI Project. Itanium c++ abi: C++ abi specification, 2024. C++ Application Binary Interface specification including name mangling and RTTI representation.

[9] Andrew Kennedy and Don Syme. Design and implementation of generics for the .net common language runtime. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1–12. ACM, 2001.

[10] LLVM Project. Llvm language reference manual, 2024. LLVM intermediate representation and code generation model.

[11] Microsoft. Generic math in .net. `https://learn.microsoft.com/en-us/dotnet/standard/generics/math`, 2025.

[12] Oracle Corporation. The java virtual machine specification. `[https://docs.oracle.com/javase/specs/jvms/](https://docs.oracle.com/javase/specs/jvms/)`, 2025.

[13] Project Lombok. Project lombok: Spice up your java. `https://projectlombok.org/`, 2024.

[14] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Professional, 4 edition, 2013.

[15] The Rust Project Developers. Monomorphization and code generation, 2024.

[16] David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor. *C++ Templates: The Complete Guide*. Addison-Wesley Professional, 2 edition, 2017.